

Visualisation avec . Évaluation finale

Dernière mise à jour : 06 nov. 2025

Sandrine LYSER

Table des matières

1	Rappel des consignes et données	3
2	Eléments de correction	4
2.1	Mise en place du projet [2 points]	4
2.1.1	Énoncé	4
2.1.2	Réponse	4
2.1.3	Question 1	4
2.1.4	Réponse 1	4
2.2	Préparation des données [3 points]	5
2.2.1	Énoncé	5
2.2.2	Commandes R	5
2.2.3	Explications	5
2.2.4	Question 2	6
2.2.5	Réponse 2	6
2.3	Visualisation [15 points]	7
2.3.1	Graphique 1	7
2.3.1.1	Énoncé	7
2.3.1.2	Commandes R	7
2.3.1.3	Graphique	7
2.3.1.4	Question 3	8
2.3.1.5	Réponse 3	8
2.3.2	Graphique 2	10
2.3.2.1	Énoncé	10
2.3.2.2	Commandes R	10
2.3.2.3	Graphique	10
2.3.2.4	Explications	11
2.3.3	Graphique 3	12
2.3.3.1	Énoncé	12
2.3.3.2	Commandes R	12
2.3.3.3	Graphique	12
2.3.3.4	Explications	13

2.3.4	Graphique 4	14
2.3.4.1	Énoncé	14
2.3.4.2	Commandes R	14
2.3.4.3	Graphique	15
2.3.4.4	Explications	16
2.3.4.5	Solution alternative	17

1 Rappel des consignes et données

À partir du jeu de données `films` vous devez effectuer une analyse reproductible et répondre aux trois questions qui vous sont posées.

Vous écrierez les commandes et les réponses aux questions dans un script R.

Le jeu de données `films`

Les données concernent les films Netflix. Le fichier `films.csv` est à télécharger depuis l'espace [Elearn](#).

Ce fichier contient les variables suivantes :

- **id_oeuvre** : identifiant du film
- **titre** : titre du film
- **pays** : pays
- **année_sortie** : année de sortie
- **genre** : genre de film
- **langue** : langue du film
- **notemoyenne** : note moyenne sur 10
- **budget** : budget du film
- **recettes** : recettes du film

Ces données ont été extraites du jeu de données `netflix_movies_detailed_up_to_2025.csv` ([voir la source](#) - Dernier accès le 06 oct. 2025).

2 Eléments de correction

2.1 Mise en place du projet [2 points]

2.1.1 Énoncé

Créez un projet RStudio et organisez votre script et vos fichiers de données de manière à ce que le projet soit reproductible.

2.1.2 Réponse

Le projet RStudio doit comporter au minimum

- un dossier pour stocker les scripts R (ou d'un autre format, Rmarkdown ou quarto par exemple) utiles pour le travail, que l'on peut nommer **scripts**
- un dossier qui contient les données source et/ou produites, que l'on peut nommer **data**
- un dossier où sont stockés les graphiques créés au cours de l'analyse, que l'on peut nommer **plots**

2.1.3 Question 1

Indiquer en 2 points en quoi un projet RStudio est utile pour la reproductibilité d'une analyse*

2.1.4 Réponse 1

Un projet RStudio est particulièrement utile pour la reproductibilité d'une analyse car il permet :

1. de centraliser tous les éléments associés à un projet (scripts, données, documentation, graphiques), ce qui garantit que l'analyse peut être reproduite à l'identique ultérieurement ou par d'autres personnes et sur différents environnements
2. d'automatiser l'organisation du travail avec une gestion facilitée des chemins d'accès et du répertoire de travail

2.2 Préparation des données [3 points]

2.2.1 Énoncé

- Chargez le jeu de données dans votre projet, **en utilisant le tidyverse**
- Vérifiez que le jeu de données respecte les bonnes pratiques de codage (corrigez si nécessaire)
- Créez une nouvelle variable calculée *profit* : $profit = recettes - budget$

2.2.2 Commandes R

```
#  
# Chargement des packages #####  
library(tidyverse)  
  
#  
# Import des données #####  
readr::read_csv2("data/films.csv") -> films  
  
#  
# Correction du nom des variables #####  
films |>  
  rename(annee_sortie = année_sortie,  
         note_moyenne = notemoyenne) -> films  
  
#  
# Création de la variable "profit" #####  
films |>  
  mutate(profit = recettes - budget) -> films
```

2.2.3 Explications

La commande `films |> rename(annee_sortie = année_sortie, note_moyenne = notemoyenne)` permet de renommer

- "année_sortie" en "annee_sortie" pour enlever les accents qui peuvent poser problème
- "notemoyenne" en "note_moyenne" pour garder une certaine cohérence dans le nom des variables, qui sont écrites en suivant la convention de nommage `underscore_separated`, qui écrit les noms de variables en minuscules avec les mots séparés par un *underscore* (`_`)

2.2.4 Question 2

Qu'est-ce-que le tidyverse ? Citez 3 packages qui en font partie.

2.2.5 Réponse 2

Le tidyverse est un ensemble de packages R conçus principalement par Hadley Wickham, pour la science des données. Ces packages partagent une philosophie commune, une syntaxe uniforme et des formats de données compatibles, afin de rendre la manipulation, la transformation et la visualisation des données plus intuitives, efficaces et reproductibles. Dans le tidyverse, les données sont organisées de manière "propre" (*tidy*) de sorte que dans le jeu de données chaque variable = une colonne et chaque observation = une ligne.

Les neuf packages principaux du tidyverse, qui sont chargés lors de l'exécution de la commande `library(tidyverse)` sont les suivants :

- `dplyr` pour la manipulation de données (filtrage, sélection, regroupement)
- `ggplot2` pour la visualisation graphique
- `tidyr` pour la mise en forme des données
- `readr` pour l'import/export de données
- `tibble` pour un format de tableau de données amélioré
- `purrr` pour la programmation
- `stringr` pour la gestion des chaînes de caractères
- `forcats` pour les variables catégorielles
- `lubridate` pour la manipulation des dates et heures

Ces packages peuvent bien entendu être chargés individuellement avec la commande `library(_nomdupackage_)`

2.3 Visualisation [15 points]

2.3.1 Graphique 1

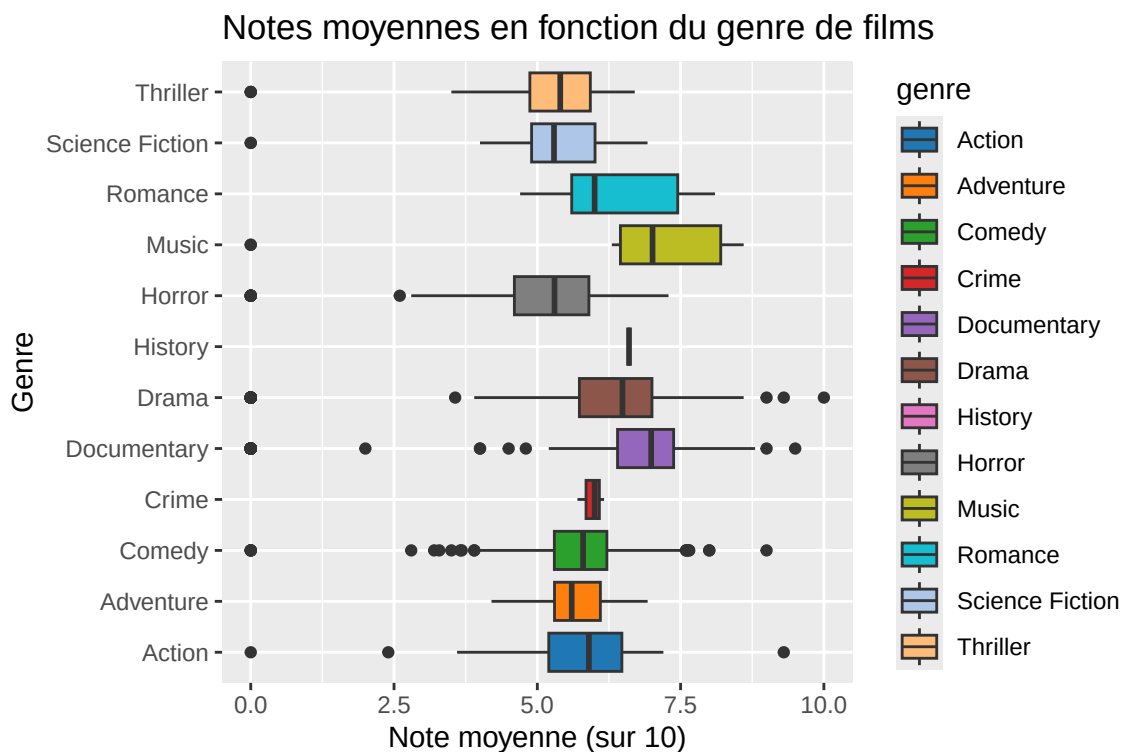
2.3.1.1 Énoncé

- Représentez la distribution des notes moyennes par genre en utilisant **un boxplot**
- Personnalisez les couleurs

2.3.1.2 Commandes R

```
ggplot(data = films) +  
  aes(x = genre, y = note_moyenne, fill = genre) +  
  geom_boxplot() +  
  scale_fill_manual(values = c("#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",  
                                "#9467bd", "#8c564b", "#e377c2", "#7f7f7f",  
                                "#bcbd22", "#17becf", "#aec7e8", "#fbb748")) +  
  labs(title = "Notes moyennes en fonction du genre de films",  
        x = "Genre",  
        y = "Note moyenne (sur 10)") +  
  coord_flip()
```

2.3.1.3 Graphique



2.3.1.4 Question 3

Décrivez le rôle de chacune des couches que vous utilisez dans ce graphique

2.3.1.5 Réponse 3

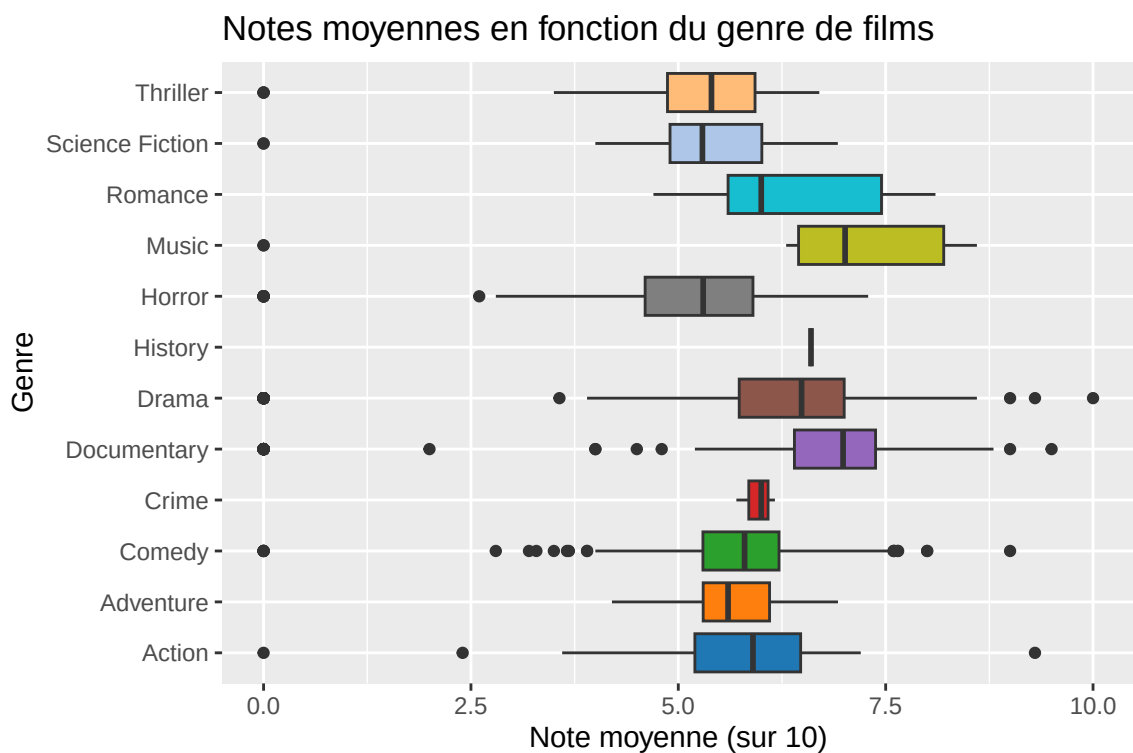
- La fonction `ggplot()` sert à initialiser le graphique. Le jeu de données à utiliser est précisé comme paramètre de la fonction
- La couche `aes()` permet de préciser les variables que l'on souhaite représenter sur le graphique. Ici sur l'axe des abscisses, on positionne le genre de films et sur l'axe des ordonnées, on positionne les notes
- La couche qui suit, `geom_boxplot()` indique le type de graphique souhaité. Ici, une boxplot
- Les couches qui sont ajoutées par la suite ne sont pas obligatoires, contrairement aux précédentes. Elles sont ajoutées pour améliorer la mise en forme du graphique et faciliter sa lecture
 - `scale_fill_manual()` définit manuellement la palette couleur utilisée
 - `labs()` sert à ajouter les étiquettes au graphique : titre principal, de l'axe des abscisses et de l'axe des ordonnées
 - `coord_flip()` effectue une rotation du graphique, ce qui permet une meilleure lisibilité des étiquettes de l'axe des abscisses

2.3.1.5.1 Informations complémentaires

- On aurait pu inverser x et y et ne pas utiliser `coord_flip()`
- On pourrait supprimer la légende qui n'apporte pas d'information complémentaire, en ajoutant la commande `theme(legend.position = "none")`

Commandes actualisées

```
ggplot(data = films) +  
  aes(x = note_moyenne, y = genre, fill = genre) +  
  geom_boxplot() +  
  scale_fill_manual(values = c("#1f77b4", "#ff7f0e", "#2ca02c", "#d62728",  
                                "#9467bd", "#8c564b", "#e377c2", "#7f7f7f",  
                                "#bcbd22", "#17becf", "#aec7e8", "#ffbb78")) +  
  labs(title = "Notes moyennes en fonction du genre de films",  
        x = "Note moyenne (sur 10)",  
        y = "Genre") +  
  theme(legend.position = "none")
```



2.3.2 Graphique 2

2.3.2.1 Énoncé

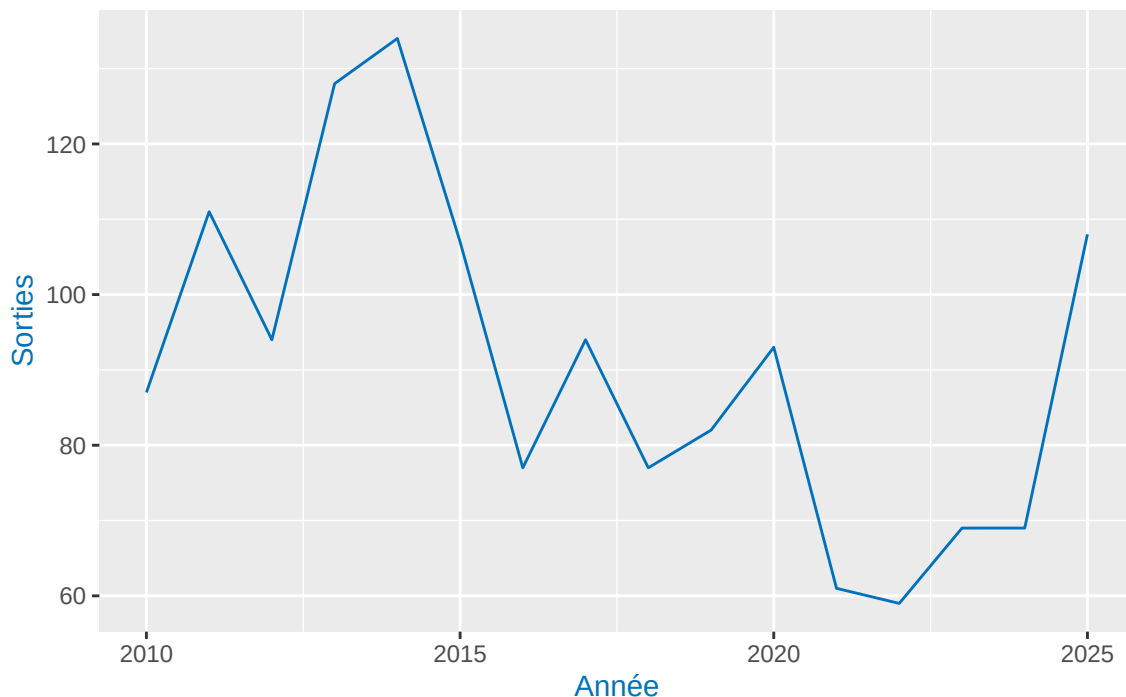
- Montrez l'évolution du nombre de films par année, en utilisant **une courbe**
- La courbe ainsi que les libellés des axes seront de couleur bleue
- Exportez ce graphique dans un fichier image (png, jpeg, etc.)

2.3.2.2 Commandes R

```
# Préparation des données : calcul du nombre de sorties par année avec count()
films |>
  dplyr::count(annee_sortie) |> # Le résultat est dans une variable nommée "n"
# Graphique
ggplot() +
  aes(x = annee_sortie, y = n) +
  geom_line(color = "#0072BB") +
  labs(title = "Nombre de sorties de films entre 2010 et 2025",
       x = "Année",
       y = "Sorties") +
  theme(axis.title.x = element_text(color = "#0072BB"),
        axis.title.y = element_text(color = "#0072BB"))
# Export du graphique
ggsave(file = "plots/SortiesParAn.png")
```

2.3.2.3 Graphique

Nombre de sorties de films entre 2010 et 2025



2.3.2.4 Explications

Avant de faire le graphique, il faut calculer le nombre de sorties par année, avec la fonction `count()`

```
films |>
  dplyr::count(annee_sortie)
```

```
# A tibble: 16 x 2
  annee_sortie     n
  <dbl> <int>
1      2010      87
2      2011     111
3      2012      94
4      2013     128
5      2014     134
6      2015     107
7      2016      77
8      2017      94
9      2018      77
10     2019      82
11     2020      93
12     2021      61
13     2022      59
14     2023      69
15     2024      69
16     2025     108
```

Il est ensuite possible d'utiliser cette variable "n", c'est cette variable que l'on positionne sur l'axe des ordonnées dans `aes()`

- Pour visualiser les données sous la forme d'une courbe, on utilise `geom_line()` en paramétrant la couleur de la courbe avec `color = : geom_line(color = "#0072BB")`
- La couleur des titres des axes des abscisses et ordonnées se définit dans la couche `theme()`, avec respectivement `axis.title.x` et `axis.title.y`. Comme on souhaite agir sur la mise en forme du texte, il faut utiliser la fonction `element_text()` et comme précédemment, la couleur est définie avec `color = : theme(axis.title.x = element_text(color = "#0072BB"), axis.title.y = element_text(color = "#0072BB"))`
- Pour sauvegarder le graphique que l'on vient de créer, au format image (ici *png*), on utilise la fonction `ggsave()`, en précisant que le fichier doit être stocké dans le dossier "plots"

2.3.3 Graphique 3

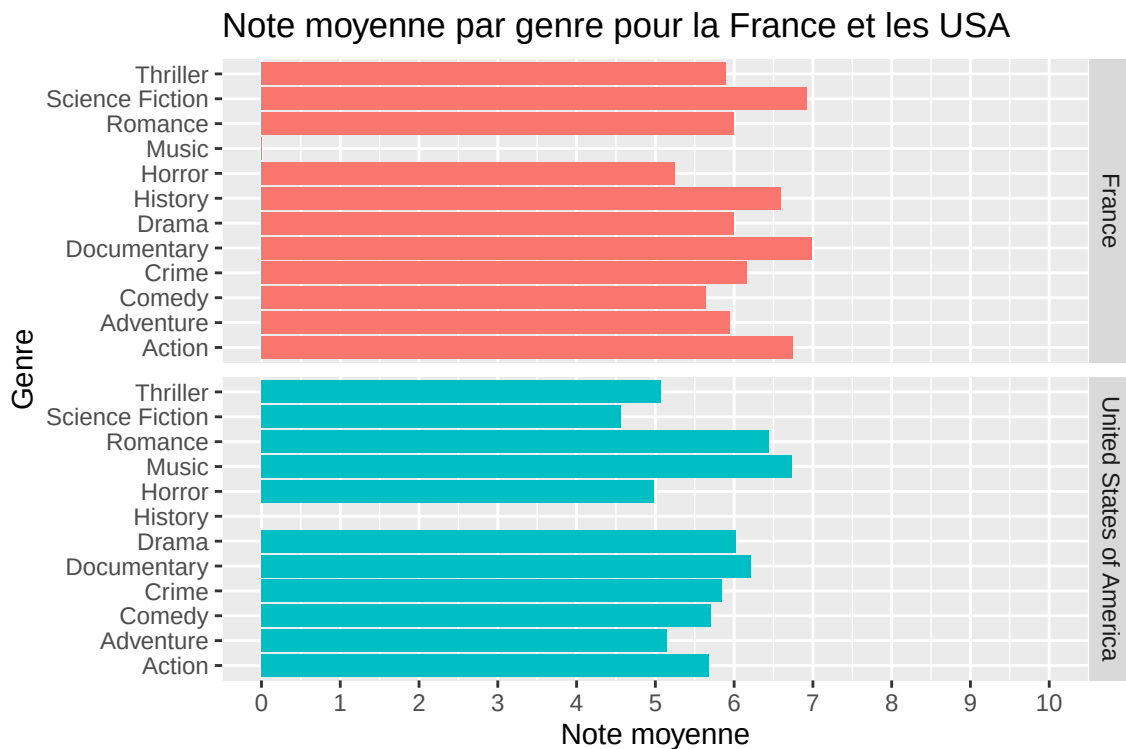
2.3.3.1 Énoncé

- Sélectionnez les films de la France et des États-Unis
- Comparez la note moyenne par genre et pays, en utilisant **un diagramme en barres**, avec une facette et une couleur pour chacun de ces 2 pays
- Faites les ajustements nécessaires pour que le graphique soit lisible et compréhensible

2.3.3.2 Commandes R

```
# Préparation des données : calcul de la note moyenne par genre et pays
films |>
  filter(pays %in% c("France", "United States of America")) |>
  group_by(genre, pays) |>
  summarise(note_genre_pays = mean(note_moyenne, na.rm = TRUE)) |>
# Graphique
ggplot() +
  aes(x = genre, y = note_genre_pays, fill = pays) +
  geom_bar(stat = "identity") +
  scale_y_continuous(limits = c(0, 10),
                     # graduations tous les 1
                     breaks = seq(from = 0, to = 10, by = 1)) +
  coord_flip() +
  facet_grid(rows = vars(pays)) +
  labs(title = "Note moyenne par genre pour la France et les USA",
       x = "Genre",
       y = "Note moyenne") +
  theme(legend.position = "none")
```

2.3.3.3 Graphique



2.3.3.4 Explications

Dans un premier temps, il faut calculer la note moyenne par genre pour la France et les USA. On va donc grouper les données sur ces 2 variables, avant de calculer la note moyenne, que l'on va appeler "note_genre_pays".

```
films |>
  filter(pays %in% c("France", "United States of America")) |>
  group_by(genre, pays) |>
  summarise(note_genre_pays = mean(note_moyenne, na.rm = TRUE))

# On aurait pu sélectionner les pays en écrivant
films |>
  filter(pays == "France" | pays == "United States of America")
```

C'est ce nouveau tableau de données qui sert de base pour le graphique.

- Dans la couche `aes()`, on positionne le genre sur l'axe des abscisses et la note sur l'axe des ordonnées. Le paramètre `fill = pays`, permet de donner une couleur différente aux barres selon le pays
- Pour faire un diagramme en barres, on utilise `geom_bar()`. Le paramètre `stat = "identity"` doit être mentionné pour bien prendre en compte la variable y précisée à la ligne précédente dans `aes()`
- L'échelle des notes allant de 0 à 10, on peut fixer les limites de l'axe des ordonnées à 0 (min) et 10 (max) avec `scale_y_continuous(limits = c(0, 10))`
- Pour améliorer la lisibilité, on fait une rotation du graphique avec `coord_flip()`
- Pour avoir une facette par pays, on utilise la fonction `facet_grid()`. Pour une meilleure lisibilité, on positionne les facettes l'une sous l'autre, avec le paramètre `rows` : `facet_grid(rows = vars(pays))`
- La commande `theme(legend.position = "none")` supprime la légende

2.3.4 Graphique 4

2.3.4.1 Énoncé

- Créez un graphique qui montre, sur **un nuage de points**, la relation entre le budget et les recettes des films pour lesquels le budget et les recettes sont tous deux positifs. Pour une meilleure visibilité : (i) modifiez l'unité des recettes et du budget pour les exprimer en millions (divisez par 1 000 000) et (ii) sélectionnez les 3 genres principaux
- Utilisez la couleur pour distinguer les années de sortie et les formes pour distinguer les genres
- Appliquez un thème prédéfini

2.3.4.2 Commandes R

```
# Préparation des données - étape 1
films |>
  # sélection des budgets et recettes positifs (>0)
  filter(budget > 0 & recettes > 0) |>
  # calcul du nombre de films selon le genre
  group_by(genre) |>
  count()
# Le résultat qui s'affiche permet d'identifier les 3 genres principaux
# Il s'agit de Drama, Comedy et Horror

# Préparation des données - étape 2
# calculs des recettes et des budgets en millions
# sélection des budgets et recettes positifs (>0)
# et des 3 genres qui ont le plus de films : Comedy, Drama et Horror
films |>
  mutate(budget_million = budget / 1000000,
         recettes_million = recettes / 1000000) |>
  filter((budget > 0 & recettes > 0) &
         (genre == "Comedy" | genre == "Drama" | genre == "Horror")) |>
# Graphique
ggplot() +
  aes(x = budget_million, y = recettes_million,
      color = annee_sortie,
      shape = genre) +
  geom_point() +
  labs(title = "Recettes des films selon leur budget",
       subtitle = "Seuls les trois genres principaux sont représentés sur le graphique",
       x = "Budget",
       y = "Recettes",
       color = "Année de sortie", # Modifie le titre de la légende couleur
       shape = "Genre de film",   # Modifie le titre de la légende shape
       caption = "(Unités : millions)") +
  ggplot2::scale_color_viridis_c() +
  # Il y a beaucoup d'années à représenter,
  # il vaut mieux utiliser une palette continue :
  # avec un gradient on voit mieux l'évolution dans le temps, la chronologie
  theme_minimal()
```

2.3.4.3 Graphique

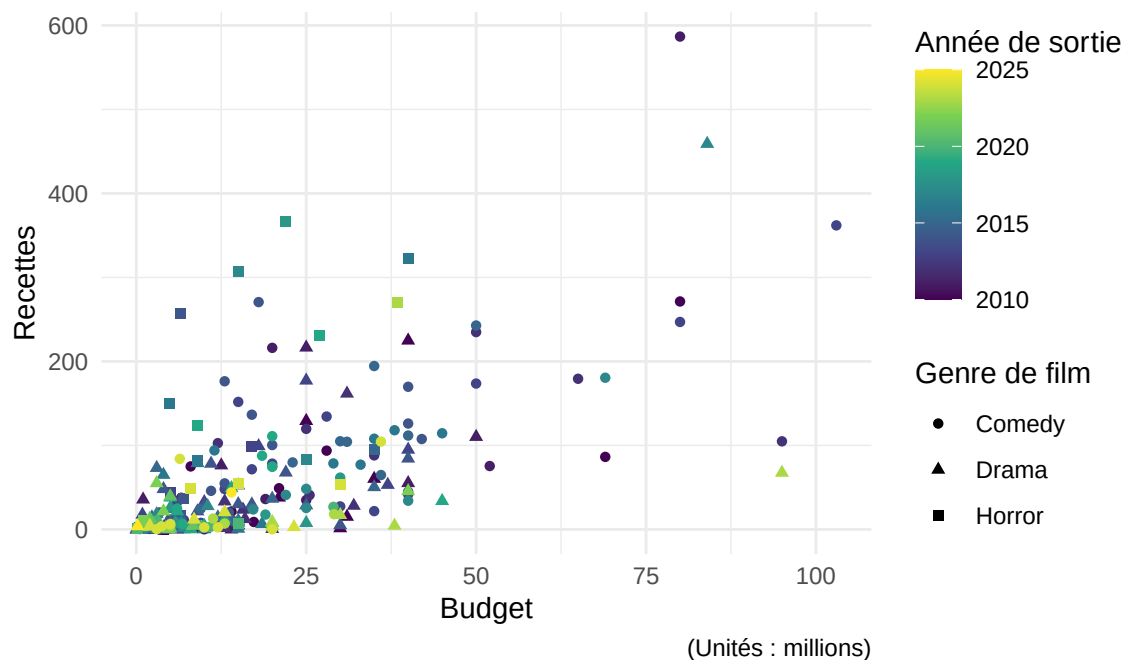
A tibble: 9 x 2

Groups: genre [9]

genre	n
<chr>	<int>
1 Action	2
2 Adventure	1
3 Comedy	113
4 Documentary	6
5 Drama	134
6 Horror	24
7 Music	2
8 Science Fiction	1
9 Thriller	5

Recettes des films selon leur budget

Seuls les trois genres principaux sont représentés sur le graphique



2.3.4.4 Explications

1. La première étape consiste, pour les films qui ont un budget et des recettes positifs, à grouper par genre, pour compter le nombre de films par genre avec la fonction `count()`

```
# Préparation des données - étape 1
films |>
  # sélection des budgets et recettes positifs (>0)
  filter(budget > 0 & recettes > 0) |>
  # calcul du nombre de films selon le genre
  group_by(genre) |>
  count()
```

```
# A tibble: 9 x 2
# Groups:   genre [9]
  genre      n
  <chr>    <int>
1 Action      2
2 Adventure    1
3 Comedy    113
4 Documentary    6
5 Drama     134
6 Horror     24
7 Music       2
8 Science Fiction 1
9 Thriller     5
```

Après affichage de ce tableau, on identifie les trois genres principaux. Il s'agit de "Comedy", "Drama" et "Horror"

2. Dans un deuxième temps, on prépare les données pour le graphique :

- On crée les nouvelles variables de budget et recettes, avec comme unité, le million : `mutate(budget_million = budget / 1000000, recettes_million = recettes / 1000000)`
- On sélectionne les films qui ont un budget et des recettes positifs (`budget > 0 & recettes > 0`) et les trois genres principaux (identifiés à l'étape 1), à l'aide d'une fonction `filter()` : `filter((budget > 0 & recettes > 0) & (genre == "Comedy" | genre == "Drama" | genre == "Horror"))`

3. Enfin on crée le graphique

- Dans la couche `aes()`, on positionne le budget (en millions) sur l'axe des abscisses et les recettes (en millions) sur l'axe des ordonnées. Le paramètre `color = annee_sortie`, permet de donner une couleur différente aux points selon l'année de sortie et le paramètre `shape = genre` donne une forme différente selon le genre
- Pour faire un nuage de points, on utilise `geom_point()`.
- La commande `theme_minimal()` applique ce thème prédéfini au graphique

2.3.4.5 Solution alternative

Une solution alternative, avec une sélection automatique des 3 genres principaux

```
# Préparation des données - étape 1
films |>
  # sélection des budgets et recettes positifs (>0)
  filter(budget > 0 & recettes > 0) |>
  # calcul du nombre de films selon le genre
  group_by(genre) |>
  count() |>
  # annuler la structuration selon les groupes
  ungroup() |>
  # tri des genres par ordre décroissant des effectifs
  arrange(desc(n)) |>
  # conservation des 3 premières lignes
  slice(1:3) |>
  # sélection de la variable "Genre" et enregistrement du résultat dans genres_top3
  pull(genre) -> genres_top3

# Préparation des données - étape 2
# calculs des recettes et des budgets en millions
# sélection des budgets et recettes positifs (>0)
# et des 3 genres qui ont le plus de films : Comedy, Drama et Horror
films |>
  mutate(budget_million = budget / 1000000,
         recettes_million = recettes / 1000000) |>
  filter((budget > 0 & recettes > 0) &
         (genre %in% genres_top3)) |>

# Graphique
ggplot() +
  aes(x = budget_million, y = recettes_million,
      color = annee_sortie,
      shape = genre) +
  geom_point() +
  labs(title = "Recettes des films selon leur budget",
       subtitle = "Seuls les trois genres principaux sont représentés sur le graphique",
       x = "Budget",
       y = "Recettes",
       color = "Année de sortie", # Modifie le titre de la légende couleur
       shape = "Genre de film",   # Modifie le titre de la légende shape
       caption = "(Unités : millions)") +
  ggplot2::scale_color_viridis_c() +
  # Il y a beaucoup d'années à représenter,
  # il vaut mieux utiliser une palette continue :
  # avec un gradient on voit mieux l'évolution dans le temps, la chronologie
  theme_minimal()
```

Recettes des films selon leur budget

Seuls les trois genres principaux sont représentés sur le graphique

